

Automated Debate Framework with Text Simplification

Project Repository:

`github.com/AdityaKarippadathUdai/NLP-Project`

Project Domain: Natural Language Processing

Core Technologies

Large Language Models, Text Simplification, Debate Generation

**AUTOMATED DEBATE FRAMEWORK
WITH TEXT SIMPLIFICATION**

Version: 1.0

Abstract

Nowadays, due to rapid social media and internet progress, access to a wide range of information. To a large extent, however, this information is expressed in complex terminology and is biased in a certain way, making it difficult to users to have full knowledge or to have a fair opinion. To get rid of this issue, we will propose a Chatbot Answer Text Simplification. The chatbot reads the input text and it is then passed through a claim extraction module. A using a will identify the most relevant pieces or key points in the text. type of the transformer-based model transformer-type Flan-T5. The little attention given to any significant claims of the concentrators. system removes information redundancy, and increases clarity. When stitching out the claims, the chatbot breaks down the claims by paraphrasing the compound sentences, transforms the complex words into the easy ones and alters their sense. It also provides Wikipedia style definitions of important entities so that the user can better understand the definitions. is unfamiliar with words and ideas. Where claims are doubtful, it has a web scraping component that has been written in. Beautiful Soup to retrieve data on a variety of Internet sites. This ensures that different sides of the issue, those in support and those against are gathered. The It removes all but a similar and relevant data filtered data is collected using similarity and relevance techniques. contextually relevant and important content. Finally the filtered information is pumped into a Mistral model and an output bal is generated. anced output. This production is composed of benefits and drawbacks of the subject and a neutral conclusion. Overall, the chatbot reduces bias, simplifies complicated information, and. provides unbiased information to empower users to make informed decisions.

1 Introduction

1.1 Background

Text simplification is an important domain in Natural Language Processing (NLP), for improving the easy access of textual content. The goal of text simplification is to represent complex sentences into simpler sentences for better readable forms without changing the original meaning. In many real-world scenarios, users often encounter complicated technical, or opinion-heavy text which turns out to be difficult to understand. This problems is significant when the content involves arguments, claims, or debates, where clarity and balance are essential. As there is a rapid growth in AI-powered language models such as FLAN-T5 and Mistral, it is now possible to build systems that not only simplify text but also analyze and validate as well as generate counter arguments. This proposed work implements a Debate-Based NLP Chatbot that first performs a claim-first pipeline in which the input text is broken into claims, simplified, analyzed for debatability, and then enriched with evidence and counterarguments to generate a balanced output.

1.2 Importance in NLP

Explain the significance of this problem within the field of Natural Language Processing.

1.3 Objectives

- Non-native English speakers might struggle with complex textual content so, the proposed models helps in Simplification and improves readability and inclusivity.
- In today's internet era a lot of misinformation is spread so, just simply presenting simplified text is not enough. Users need balanced perspectives which includes supporting and opposing arguments.
- Traditional NLP systems provide outputs without providing justification. This proposed work enhances explainability by: Extracting claims Providing evidence Generating counter arguments
- The system consists of multiple NLP tasks such as text simplification, NER, debatability classification, information retrieval, and natural language generation into a single pipeline, enabling context-aware text processing.
- The applications of the system include in educational tools, news analysis platforms, debate assistants, and policy understanding systems to improve accessibility.

1.4 Scope

The objective of the proposed work is to design and implement a Debate-Based Text Simplification Chatbot which the people with difficulty to understand the language used in the text by enhancing the readability, interpretability, and analytical depth of textual content. The proposed system mainly aims to not only simplify the given complex text but also provide a structured understanding of the arguments by presenting it within it by generating balanced viewpoints. The core component of the system is claim-first pipeline architecture, where the input text is first decomposed into smaller, meaningful units called claims. This approach mainly helps the system to process statement independently by enabling precise simplification, analysing, and reasoning through breaking down large paragraphs into individual ones. claims helps in improving both interpretability and processing efficiency. After the claims have been extracted, each claim is passed through the text simplification module which is developed using FLAN-T5 model. This module help in rewriting the complex sentences into simpler ones by preserving the original meaning and making the content more easy for the users to understand. To further enhance the understanding we have integrated NER to identify key entities such as people, organizations, locations, and important terms within each claim. These entities are supported by an external knowledge sources which is Wikipedia API to which provides a short definitions and descriptions. This additional layer of information helps users better understand domain specific terms. Another important aspect within the scope is the system’s ability of the system to tell whether claim is debatable or not by making use of a classification mechanism.

The system detects if a statement can be argued or not by considering multiple perspectives. The claims which are identified as debatable are then processed to retrieve relevant supporting and opposing information from external sources. A counterargument generation module is also included in the architecture where models Mistral are used to generate contextually relevant opposing viewpoints. The proposed work help in presenting a balanced perspective by considering both sides of an argument rather than only providing a single biased interpretation. Finally, the system combines all the simplified claims which are enriched entity information, supporting evidence, and counterarguments into a structured output format, which is presented to the user by an interactive interface.

2 Literature Review

2.1 Existing Work

Text simplification is a major Natural Language Processing (NLP) task which simplifies and make more understandable complex text, without losing, its original meaning. Transformer based models have become popular with the development of deep learning. they have made great gains in the are The Text-to-Text Transfer Transformer (T5) framework introduced a unified approach where all NLP tasks are treated as text generation problems, making models to perform simplification and transformation tasks by the effectively using natural language prompts [1]. This paradigm allows easy adaptation across multiple tasks without requiring task-specific fine tuning or training new llm for it.

Along with simplification, argument mining is an important activity in claim identification and reasoning in textual information. It is about seperating out useful argumentative claims like supporting evidence and claims. Research in this field highlights the importance of structured claim extraction for enabling downstream reasoning and debate systems [4]. These methods are the major path provider for systems which aim to analyze and generate arguments.

Natural Language Inference (NLI) is a main thing to understand the logical relationships between sentences. The Multi-Genre Natural Language Inference (MNLI) dataset is a large-scale benchmark to train models which determine s entailment, contradiction, or neutrality between sentence pairs [3]. This is used for modern NLP systems to do classification tasks by understanding semantic relationships instead of relying completely on words matching.

Recent advancements have shown that the NLI models can be used for zero-shot classification by converting classification tasks as inference problems. In this , candidate labels are made as natural language hypotheses, and the model decides which label suits best with the input text based on entailment scores [2]. This removes the need for task-specific labeled datasets and allows models to generalize content.

To make system efficient and scale it further, lightweight DistilBERT is been used. DistilBERT uses knowledge distillation techniques to compress larger models, bert, which is encoder only model, without losing their performance. It reduces model size and computational requirements and also maintains strong language understanding capabilities, making it suitable for real-time applications [5].

modern NLP systems are relying on outer sources to increase accuracy and reliability. Retrieval-Augmented Generation (RAG) combines information retrieval with language models to provide grounded outputs [6]. Similarly, Dense Passage Retrieval (DPR) gives efficient retrieval of relevant evidence from large datasets [7]. These approaches shows

importance of combining retrieval mechanisms with language models to overcome limitations.

Named Entity Recognition (NER) is used to detect important entities like persons, organizations, and locations in given text. Neural architectures for NER have improved entity detection accuracy, making systems to glitter textual data with contextual information [8].

3 Methodology

3.1 System Architecture

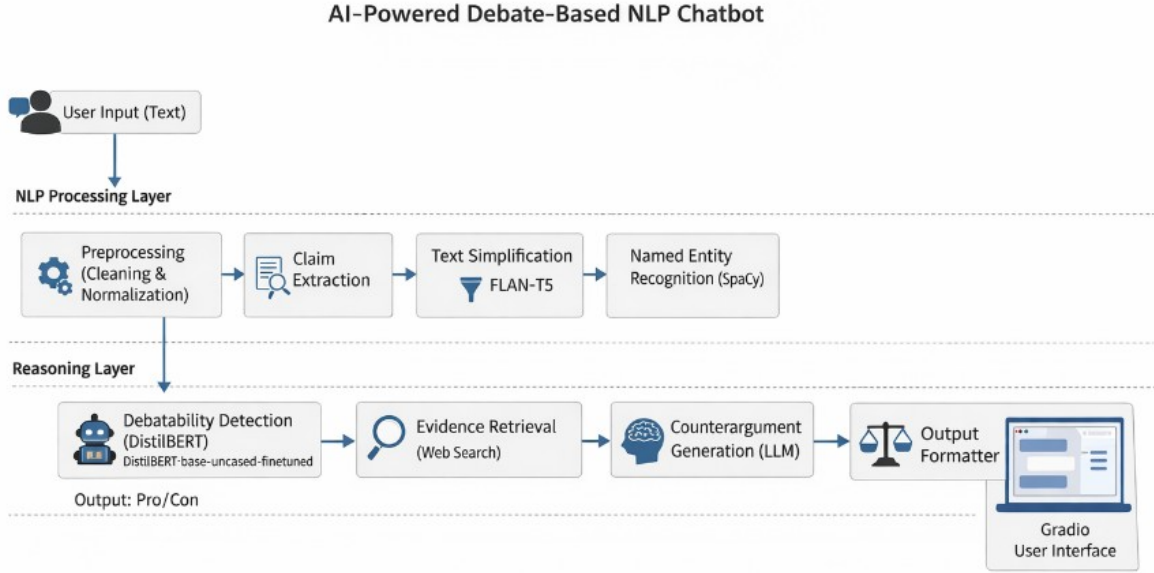


Figure 1: Overall System Architecture Diagram

The system follows a modular claim-first pipeline architecture, where input text is made into individual claims and processed through different Natural Language Processing (NLP) modules. The architecture is divided into four major layers:

- **Input Layer:** we take raw user text through a Gradio-based interface, handling different input types from short opinion statements to complex arguments.
- **NLP Processing Layer:** The claims in the user input are extracted and simplified by the flan t5 models and the entities in the claim are combined with a wikipedia definition to improve the user understanding of the text .
- **Reasoning Layer:** We use a distill bert model with zero shot to classify the claims as debatable or not . Then the system uses beautiful soap to retrieve content from

the web about the debatable claims to get the need context for the llm to generate supporting and opposing arguments.

- **Output Layer:** All the outputs from all the layers are shown in the gradio ui so that the user can see the output content from each module like the extracted and simplified claims, the debatability of claims, the webscraped contents and the final mistral arguments and conclusion.

This modular design by splitting the project into 7 modules ensures scalability, flexibility, and ease of integration of different models and components. We can develop one module then test it and then move to the next one. Each module communicates through interfaces, allowing individual components to be upgraded or replaced without affecting the overall system. This modular design by splitting the project into 7 modules ensures scalability, flexibility, and ease of integration of different models and components. We can develop one module then test it and then move to the next one. Each module communicates through interfaces, allowing individual components to be upgraded or replaced without affecting the overall system.

3.2 Phases of the Workflow

The project can be divided into 10 phases which are done one after another

1. **Data Ingestion:** The user gives input text through the Gradio-based web interface. The text may have complex sentences, biased opinions, facts, or arguments.
2. **Preprocessing:** The input text is cleaned and normalized. We remove the extra whitespaces or any special characters and symbols like percentages which is needed to ensure the model receives a proper input.
3. **Claim Extraction:** The preprocessed text is split into individual atomic claims with flant5. Since a sentence may contain many claims we use flant5 to extract the individual claims that are present in the sentence.
4. **Claim Simplification:** Each extracted claim is sent to the FLAN-T5 model for simplification. Complex or verbose claims are rephrased into clearer, more concise forms while preserving their original semantic grammatical meaning, improving understanding by both models and end users.
5. **Entity Recognition and Enrichment:** Named entities such as persons, organizations, dates, and locations are extracted from each claim using SpaCy. The Wikipedia API is then used for each recognized entity to get a definition, to improve the user's understanding of the claim's subject matter the wikipedia definition is taken as summary in 2 sentences and added in brackets.

6. **Debatability Detection:** Each simplified claim is classified as either debatable or non-debatable using a classification that combines DistilBERT and zero shot. In cases where distillbert is unable to classify properly then we use a gemini API to access gemini 2.5 flash to do the classification. Non-debatable claims are not processed later , ensuring that we generate counter arguments for debatable points since
7. **Evidence Retrieval:** For each debatable claim, relevant supporting and opposing evidence is retrieved from the internet using beautiful soap. Web scraped content is then ranked by relevance and similarity to the claim before being passed to the counter argument generation stage.
8. **Counterargument Generation:** The Mistral large language model generates coherent counterarguments based on the simplified claim and retrieved evidence which is given to llm as evidence. The model is give the evidence retrieved from the internet as well as it uses its own knowledge to generate the supporting and opposing arguments and a balanced conclusion.
9. **Balanced Synthesis:** The system aggregates simplified claims, supporting arguments, retrieved evidence, and generated counterarguments into a unified structured output. The synthesis step ensures the response is balanced and does not favour any single perspective.
10. **Output Presentation:** The final structured result is rendered and displayed through the Gradio-based user interface in a readable, organized format that clearly separates supporting arguments from counterarguments.

3.3 Model Descriptions

The system integrates four state-of-the-art NLP models, each specialized for a particular task within the pipeline. The following subsections describe the architecture, working mechanism, and role of each model in detail.

3.3.1 FLAN-T5

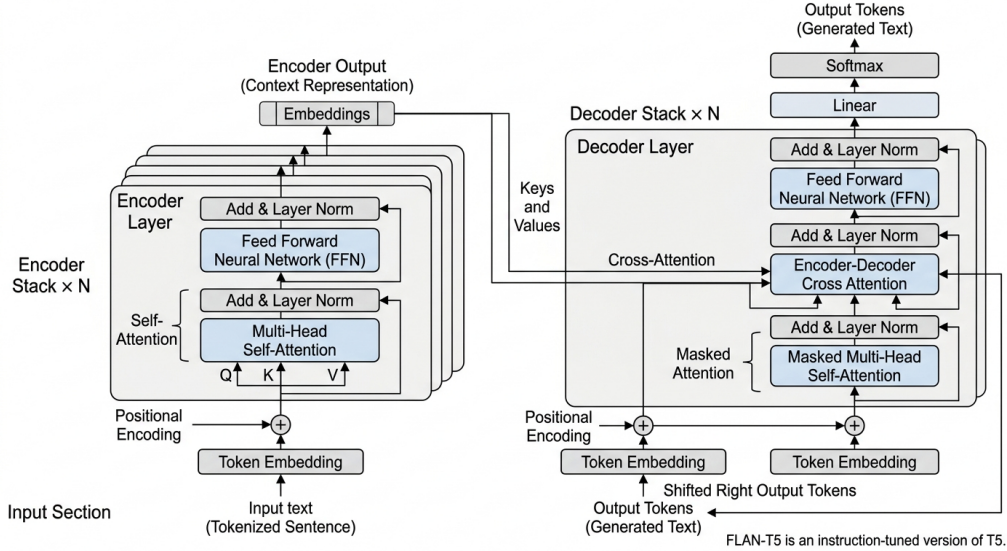


Figure 2: FLAN-T5 Encoder-Decoder Architecture

FLAN-T5 (Fine-tuned Language Net – Text-to-Text Transfer Transformer) is a transformer-based encoder-decoder model developed by Google Research. It is built based on the T5 architecture and is specifically designed for text-to-text tasks, meaning every NLP problem whether summarization, translation, classification, or simplification is framed as mapping an input text string to an output text string. The FLAN T5 model is trained on a diverse set of 1000 tasks so it can perform well even on unseen tasks as well. The model can reason well so we use it for claim extraction and simplification as the FLAN T5 can reason well and understand the meaning of the text to get the valid claims and simplify them.

Architecture: FLAN-T5 has a standard encoder-decoder transformer architecture. The encoder processes the input token sequences and produces a set of contextualized hidden representations using multi-head self-attention mechanism. Each attention head then independently computes query, key, and value projection, allowing the model to attend to different parts of the input at the same time. The decoder then generates the output sequence token-by-token, using both previously generated tokens (with masked self-attention) and the encoder’s output (by cross-attention). Relative positional embeddings

are used to encode token position , making the model more robust to sequences of varying length .

Instruction Tuning: What separate the FLAN-T5 from normal T5 is its instruction tuning. The model is fine-tuned on a large collection of tasks expressed as natural language instructions about a 1000 tasks, enabling it to generalize to new tasks described in plain language without requiring task-specific training data. This makes FLAN-T5 highly adaptable — for text simplification, the model receives a prompt such as “*Simplify the following claim:*” followed by the input text, and generates a simplified paraphrase.

Working in This System: Each extracted claim is givent to the model as a natural language instruction and fed to the FLAN-T5 encoder. The decoder generates a simplified version of the claim by repeatedly sampling the most probable next token conditioned on the encoder’s context. Beam search is used to improve output quality by checking multiple sentences at same time and choosing the best one in our project we use 5 as beam. The simplification reduces complexity of words and grammar used and sentence length while preserving the core argument and meaning of sentence for user to understand, improving the accuracy of downstream classification and generation.

Why FLAN-T5: FLAN-T5 was chosen because its instruction tuned Which allows it to perform well on simplification tasks without fine-tuning the model for simplification task, reducing training resources models like t5 would need to trained again for the task. Its encoder-decoder architecture is inherently well-suited to sequence-to-sequence transformations,

3.3.2 DistilBERT (Debatability Detection)

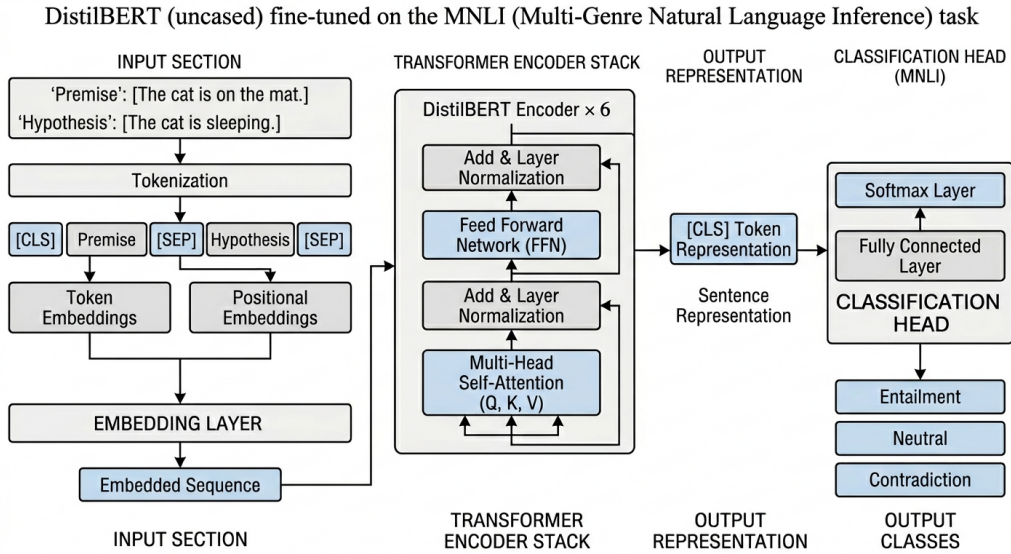


Figure 3: DistilBERT Classification Architecture

DistilBERT (Distilled Bidirectional Encoder Representations from Transformers) is a compressed version of BERT developed by Hugging Face. It was trained using knowledge distillation a model compression technique where a smaller student model is trained to replicate the behaviour of a larger teacher model like BERT but still showing about 97% of BERT’s performance while being 40% smaller in size and 60% faster. This is useful as we don’t need more GPU memory to run distill bert.

Architecture: DistilBERT is an encoder-only transformer model which consists of six transformer layers. Each layer contains a multi-head self-attention mechanism followed by a add and normalize then feed-forward network with layer normalization and residual connections. The model then processes input as a sequence of tokens generated by wordpiece tokenizer, with a special [CLS] token prepended to every input which is used to get meaning of the entire sentence as it contains no meaning on its own.

Knowledge Distillation Training: During distillation, DistilBERT is trained on a combination of three objectives: (1) *soft target cross-entropy loss* matching the teacher’s output probability distribution (2) *hard label cross-entropy loss* — matching ground-truth labels; and (3) *cosine embedding loss* — so that the student’s hidden state representations are somewhat similar to those of the teacher. This multi-objective training ensures the student not only produces correct predictions but also mimics the internal representations of the larger teacher model like BERT.

Working in This System: Each simplified claim is tokenized and fed to DistilBERT. The resulting [CLS] embedding captures whether the claim expresses a subjective stance (debatable) or an objective fact (non-debatable). We use zero shot instead of fine tuning the model for classifying the statement as debatable or not we give it the needed markers. Claims classified as non-debatable are not taken for the counterargument generation, as for non-debatable topics we cannot generate supporting and opposing statements.

Why DistilBERT: DistilBERT’s smaller size of about 300mb makes it suitable for local use without requiring high GPU vRAM, while its accuracy ensures that the claims are classified accurately. Its bidirectional attention allows it to fully consider both left and right context for every token at the same time, which is needed for understanding complex biased or strong opinion sentences.

3.3.3 Mistral (Counterargument Generation)

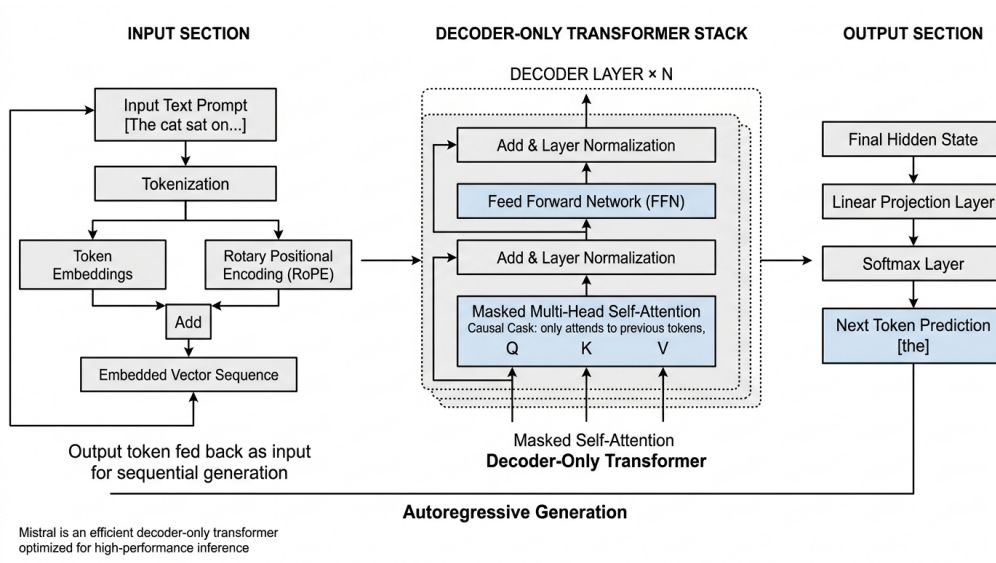


Figure 4: Mistral Decoder Architecture with Grouped-Query and Sliding Window Attention

Mistral 7B is a large autoregressive llm model developed by Mistral AI. Despite having only 7 billion parameters, it outperforms certain larger models on multiple benchmarks, owing to architectural innovations that uses both transformers and rnn that improve both efficiency and reasoning capability.

Architecture: Mistral uses a decoder-only transformer architecture similar to GPT-like model , where the model processes input tokens and generates output tokens sequentially using masked self-attention. Key architectural innovations in mistral 7b are:

- **Grouped-Query Attention (GQA):** It Reduces memory consumption during inference by sharing key-value heads across multiple query heads, which reduces the key value cache footprint without reducing the accuracy.
- **Sliding Window Attention (SWA):** Limits a token's attention to a fixed size token window rather than the full senetnce, which reduces the quadratic complexity of standard attention linear for long input sentences.
- **Rolling Buffer Cache:** Stores only the most recent key-value states within the attention window, for better text generation .

Feed-Forward and Normalization: Each transformer layer in Mistral uses a SwiGLU (Swish-Gated Linear Unit) activation function in the feed-forward network instead of the conventional ReLU, which has better gradient flow compared to relu and

improved training results. RMSNorm (Root Mean Square Layer Normalization) is applied to each sub layer rather than in the end, after the pre-normmalization architecture which improves training stability for large models.

Prompt-Based Counterargument Generation: In this system, Mistral is prompted with a structured input containing the simplified claim and the web scraped content obtained from web search which was filtered for similarity. The model is prompted to generate 5-6 supporting arguments and 5-6 opposing argumnets and a 5 sentence conclusion which is neutral. Token generation is controlled using top sampling with a temperature parameter to balance coherence and diversity in the output.

Working in This System: Once a claim is classified as debatable and evidence is retrieved, the claim, entity context, and evidence are assembled into a prompt template. Mistral processes this prompt autoregressively, generating a counterargument token-by-token until a stopping condition (end-of-sequence token or maximum length) is reached. The mistral model role in this system is that of a argument generator which uses content taken form internet as context to get 3 types of content supporting opposing and neutral.

Why Mistral: Mistral's efficient attention mechanism allow it to process longer contexts about 7000 tokens which is needed to handle content from the web containing claims as without enigh context the model will not be able to generate proper arguments, it may be biased. The mistral model can alsio reason pretty well which is needed as the mdoel needs to understand if the web content is supporting or opposing the claim . Also compared to models like lama which have token limit of 4096 mistral has about 7000, which allows for more detailed output . Mistral model is 4 bit quantized so that we can run it locally , in quantization we compress the 32 bit vctor to 4bit to reduce space but preserve the knowledge.

3.3.4 SpaCy (Named Entity Recognition)

SpaCy is a python library which contains functionalities related to NLP. It helps in finding tokenization, pos tagging, dependency parsing, and NER. It is Computationally optimized to perform NLP tasks efficiently. Pipeline Architecture: The data in SpaCy processed in a sequential pipeline of annotators. Each annotator adds linguistic annotations to the document object in place.

Pipeline Architecture: In the proposed system the following things are considered: the tokenizer which helps in splitting raw text into tokens the tagger performs assigning pos tags, the dependency parser performs identification grammatical relationships and the NER identifies named entities.

NER Model Architecture: NER is implemented as a transition based parser using neural networks. It works using a stack-based state machine where every state is encoded as the current partially annotated document, and the model predicts the next transition

action: • **SHIFT**: Places the token from the input buffer onto the stack. • **REDUCE**: Pops the top most element of the stack. • **ENTITY** transitions (B, I, L, U): Tags the current token as the beginning, inside, last, or a unit span of a named entity. The neural network makes use of a token-level feature encoder that maps every token to a dense embedding capturing its local context. These embeddings are then passed to a feedforward network which assigns a score to every transition action.

Entity Types Recognized: Entity Types Recognized: The en core web sm or en core web trf model recognizes the spanning entity categories which include PERSON, Organizations (ORG) Geopolitical Entities(GPE), DATE (absolute or relative dates), and several others. These entities help the Wikipedia API to enrich step by determining which entities to query for contextual information.

Working in This System: The simplified claims are passed through the SpaCy pipeline. The NER component identifies all named entities within the claim and returns their text spans along with their predicted entity types. For every recognized entity we query the Wikipedia API to retrieve a short contextual description, which is then appended to the claim’s metadata. This context is later helpful for Mistral to generate better-informed counterarguments which are grounded in real-world knowledge.

3.3.5 Zero-Shot Classification (Debatability Detection)

Zero-shot classification is a Natural Language Processing technique which is used by a model to classify text into predefined labels without being fine tuned to classify on those labels. In this project, zero-shot classification is used for debatability classification to determine whether a given claim is debatable or not. In our project we use distillbert which is designed for nli as it has an encoder-only architecture; it is more suited for classification as it understands the meaning in the text. Model like distill bert trained on mnli can understand the relationship and similarity between sentences which is used along with zero shot by giving the model certain markers so that the model can classify the claim as debatable or not.

Underlying Principle: Zero-shot classification is formulated as a Natural Language Inference (NLI) problem. The input claim is treated as the premise, and each label is converted into a hypothesis using a template such as:

“This statement is {label}.”

The model predicts the relationship between the premise and hypothesis as:

- **Entailment:** The hypothesis logically follows from the premise.
- **Neutral:** The hypothesis is neither confirmed nor contradicted by the premise.
- **Contradiction:** The hypothesis is logically inconsistent with the premise.

For each candidate label l_i , the model produces a probability distribution over the three NLI classes. The entailment probability $P(\text{entailment} \mid \text{premise}, h_i)$ is extracted and used as the confidence score for label l_i . The predicted class $\hat{l}$ is the label whose hypothesis yields the highest entailment score:

$$\hat{l} = \arg \max_{l_i \in \mathcal{L}} P(\text{entailment} \mid \text{claim}, h_{l_i})$$

where $\mathcal{L} = \{\text{Debatable}, \text{Not Debatable}\}$ is the label set and h_{l_i} denotes the hypothesis generated from label l_i .

Model Architecture: DistilBERT is a lightweight transformer-based encoder model derived from BERT through knowledge distillation. It retains the core transformer structure while reducing model size and inference latency. It consists of:

- 6 transformer layers (reduced from BERT’s 12 layers)
- Multi-head self-attention mechanisms
- Feedforward neural networks
- Residual connections and layer normalization

The input sequence is structured as:

$$[\text{CLS}] \text{ Claim } [\text{SEP}] \text{ Hypothesis } [\text{SEP}]$$

Each input token x_i is taken as the sum of its token embedding, segment embedding, and positional embedding:

$$\mathbf{e}_i = \mathbf{E}_{\text{token}}(x_i) + \mathbf{E}_{\text{segment}}(x_i) + \mathbf{E}_{\text{pos}}(i)$$

Within each transformer layer, the multi-head self-attention mechanism computes, for each head k :

$$\text{head}_k = \text{Attention}(QW_k^Q, KW_k^K, VW_k^V) = \text{softmax}\left(\frac{QW_k^Q (KW_k^K)^\top}{\sqrt{d_k}}\right) VW_k^V$$

where Q, K, V are the query, key, and value matrices derived from the input, $W_k^Q, W_k^K, W_k^V \in \mathbb{R}^{d_{\text{model}} \times d_k}$ are learned projections, and $d_k = d_{\text{model}}/h$ is the per-head dimension for h attention heads. The outputs of all heads are concatenated and linearly projected:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

Each attention sublayer is followed by a position-wise feedforward network (FFN) applied

independently to each token position:

$$\text{FFN}(\mathbf{x}) = \max(0, \mathbf{x}W_1 + \mathbf{b}_1) W_2 + \mathbf{b}_2$$

Both sublayers employ residual connections and layer normalization:

$$\mathbf{x}' = \text{LayerNorm}(\mathbf{x} + \text{Sublayer}(\mathbf{x}))$$

The output corresponding to the [CLS] token, $\mathbf{h}_{[\text{CLS}]} \in R^{d_{\text{model}}}$, serves as the pooled sentence-pair representation and is passed through a linear classification head:

$$\mathbf{z} = \mathbf{h}_{[\text{CLS}]} W_c + \mathbf{b}_c, \quad W_c \in R^{d_{\text{model}} \times 3}$$

$$P(\cdot \mid \text{premise, hypothesis}) = \text{softmax}(\mathbf{z})$$

to produce probabilities over the three NLI classes.

Knowledge Distillation: DistilBERT is obtained by distilling BERT using a training objective that combines three losses. Let $\mathbf{p}^T$ and $\mathbf{p}^S$ denote the soft probability distributions of the teacher (BERT) and student (DistilBERT) respectively, computed at temperature τ . The distillation loss is:

$$\mathcal{L}_{\text{KD}} = - \sum_i p_i^T \log p_i^S, \quad \text{where } p_i = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)}$$

This is combined with a masked language modelling loss $\mathcal{L}_{\text{MLM}}$ and a cosine embedding loss $\mathcal{L}_{\text{cos}}$ that aligns the hidden-state directions of teacher and student:

$$\mathcal{L}_{\text{cos}} = 1 - \cos(\mathbf{h}^S, \mathbf{h}^T) = 1 - \frac{\mathbf{h}^S \cdot \mathbf{h}^T}{\|\mathbf{h}^S\| \|\mathbf{h}^T\|}$$

The total training objective is:

$$\mathcal{L} = \alpha \mathcal{L}_{\text{KD}} + \beta \mathcal{L}_{\text{MLM}} + \gamma \mathcal{L}_{\text{cos}}$$

Labels Used in This System: For debatability detection, the system uses the following labels:

- **Debatable:** The claim is a opinion or prediction or a biased statemnt on which we can argue about make statements which are opposing the claim , they are not univiersla truths.
- **Not Debatable:** The claim is a universal truth or a verifiable fact on which we cannot deny or find opposing evidence as its true

Each claim is evaluated against these labels using the zero-shot classification pipeline, and the final predicted label is determined by Equation (1).

Working in This System: After claim extraction and simplification, each claim is sent to the zero-shot classifier using distillbert. The model evaluates whether the claim is debatable by comparing it with predefined labels. Only claims classified as debatable are forwarded to subsequent modules such as evidence retrieval and counterargument generation. Non-debatable claims are filtered out, improving efficiency and relevance.

Why Zero-Shot Classification: Zero-shot classification is chosen because it allows us to use the distillbert model to classify the claim as debatable or non-debatable based on the markers. This is useful for debatability detection, where labelled data is limited. Using a pre-trained DistilBERT MNLI model ensures strong generalization, faster deployment, and reduced computational cost compared to training or fine tuning. This allows us to create custom classes to use classification based on our needed tasks.

3.4 Data Preprocessing Steps

Data preprocessing is done to ensure that there are no unwanted noise or irrelevant content in the input we give to our models as they may produce wrong outputs or fail to classify properly. The preprocessing pipeline is composed of seven sequential steps:

1. **Text Cleaning:** We use `re` to remove unwanted html content like tags ads url and other character which is the output from web scraping using beautiful soap as it retrieves html web pages. We also remove unwanted punctuation and whitespace which may result from webscraping by reducing them into one. This is done to ensure that the noise from webscraping does not get passed to the mistral model as this may cause model to generate poor results.
2. **Normalization:** Unicode characters are converted to ASCII value. quotes and typographic punctuation are converted to standard ASCII forms. Text casing is preserved at this stage to support NER accuracy, as capitalized letter are useful to identify Entities.
3. **Tokenization:** The cleaned text is then split into tokens by using SpaCy's tokenizer, this can deal with hyphens contractions punctuations by language rules.
4. **Stopword Removal:** Common words that carry little semantic content (articles, conjunctions, and common prepositions) are optionally removed depending on the module. Stopword removal is applied selectively — it is used in evidence retrieval queries to improve search precision but is not applied prior to model inference, where grammatical context is important.

5. **Sentence Segmentation:** The token stream is segmented into individual sentences using SpaCy’s sentence boundary detector, which uses set of rules rather than punctuation to split sentences. Sentence segmentation outputs the basic sentence from which claims are subsequently extracted as multiple claims may exist in a sentence at same time.
6. **Encoding:** Text is converted into numerical embedding vectors using the tokenizer needed with each model as each models need embedding based on the vocabulary it was trained on. FLAN-T5 uses a SentencePiece tokenizer DistilBERT uses a WordPiece tokenizer Mistral uses a Byte-Pair Encoding (BPE) Tokens are converted to integer IDs and padded or truncated to the model’s maximum input length.
7. **Noise Handling:** Incomplete sentences, single-word inputs, and small queries which dont meet the character limit are removed as these do not provide any relevant information.

3.5 Justification for the Selected Approach

The claim-first modular pipeline is selected over end-to-end monolithic architectures for several principled reasons.

Granularity and Interpretability: By splitting input into individual claims , the system got fine-grained processing which is impossible for normal paragraph level. every claim is simplified , classified and countered independently by which our system can be neutral and transparent

Modularity and Replaceability: Every component of pipeline like simplification, NER , classification , retrieval and generation is separated behind a clean interface. any module can be upgraded, swapped without any modifications in pipeline . For instance DistilBERT’s classifier could be replaced with a more powerful model as improved debatability datasets become available

Task-Specialized Models: Instead of depending on a single model to do all tasks , our systems has different modules to make model optimized for each sub task. FLAN-T5’s instruction tuning makes it good for simplification, DistilBERT’s potentiality makes it ideal for binary classification. The reasoning capacity of Mistral is efficient for complex text creation. Finally SPaCy’s pre-trained NER pipeline makes entity recognition fast

External Knowledge Integration: By integrating Wikipedia-based entity definition and getting evidences by searchin , system is not limited for parameters on which the model is pre-trained. This makes the system to reason about latest events , domain specific names.

Scalability: The module design allows individual components to scale independently based on load increased to balance. If the user request increases we can easily do repli-

cation and scale out. for example Highly delaying models like Mistrell can sometimes can be dedicated some inference servers, while lightweight components like SpaCy NER continue runs

This aproach is taken as it is modular , interpretable, scalable and capable to combine multiple NLP components which fits for real world deployment

4 Implementation Details

4.1 Programming Language and Libraries

We have implemented the project using python as in python we have many opensource libraries that support ai and NLP and data processing , python also has libraries like hugging face and transformer library that give us the option to download llm models needed for our project. For web scraping python modules like beautiful soap enable us to use web api to get relevant content from the web.

The following libraries and frameworks are used in the project:

1. HuggingFace Transformers: This library is used to load and utilize pre-trained transformer models such as FLAN-T5 for text simplification, DistilBERT for debatability detection, and Mistral for counterargument generation. To load transformers and download them from the web we use these libraries , the hugging face libraries allows us to download llm models from the huuging face cloud which gives us to various models and finetuned version that are available in the cloud. For tokenizer needed for the specific models they are accesible through hugging face and the transformer libraries. It provides efficient APIs for tokenization, model inference, and pipeline creation.

2. PyTorch: PyTorch serves as the backend deep learning framework for running transformer models. It enables efficient tensor computations and supports GPU acceleration for faster inference.

3. SpaCy: SpaCy is used for Named Entity Recognition (NER). It provides pre-trained models that can identify entities such as persons, organizations, and locations with high accuracy and speed.

4. NLTK (Natural Language Toolkit): NLTK is used for basic preprocessing tasks such as tokenization, stopword removal, and sentence segmentation.

5. Requests / HTTP Libraries: These are used to interact with external APIs such as the Wikipedia API for entity enrichment and web-based evidence retrieval.

6. Gradio: Gradio is used to build the user interface for the system. It allows users to input text and receive processed outputs in real time through an interactive web interface.

7. JSON and Data Handling Libraries: Standard Python libraries such as `json`, `re`, and `pandas` are used for data manipulation, formatting, and preprocessing.

4.2 Hardware / Software Environment

We have designed the system to run on local systems like laptops without heavy gpu support.

1. Operating System: The project was implemented on a ubuntu linux system as it has support for gpu acceleration and natively supports libraries in python for ai and deep learning and the environment management is more simpler.

2. Processor (CPU): We used An i7 multi core processor with 12 cores .

3. GPU: Then we used NVIDIA GPU to load models and perform computations as models work faster with GPU with CUDA support.

4. RAM: We need atleast 10-12 GB ram and atleast recommended is 16gb To run this project.

5. Storage: The project took 10GB storage as models like mistral take about 4GB and large number of python packages take up space in virtual environment.

6. CUDA and Drivers: For efficient use of GPU we use CUDA and drivers to allow acceleration to run the needed models for faster results.

7. Development Environment: The project was developed and tested in VS code.

8. Package Management: A python virtual environment is used to manage needed packages so that the package version needed for the package do not clash with the global python in the system.

9. Deployment Environment: The project is deployed locally by using gradio interface to take user input and visualize the outputs from each module so that we can understand working of each module.

10. Model Execution: Mistral model is 4 bit quantized form gguf to run locally , which means the weights are compressed while preserving the knowledge which allows to use the model with less VRAM.

This environment ensures that the system remains flexible, scalable, and capable of running efficiently across different platforms.

5 Results and Analysis

The proposed model is calculated on a small input para that has different facts , controversies in different categories like AI, astronomy etc. the system can work well through claim extraction and gave good structure and meaning to output

first , input is checked and claim extract model finds three different claims (1) AI will replace human jobs (2) NASA launched the james web in 2021 (3) global renewable energy investment reached 500billion in 2023. This says that our system was able to separate sentences into different

next claim wise simplification module can manage each claim separately. from typical

sentences with named entities it highlights output by add the meaning of that special word by Named Entity recognition and Wikipedia. This shows that combining transformer can enhance output

debatable detection module then classifies claim as debatable or non- debatable. system finds the claim about AI replacing jobs as debatable correctly while NASA claim as non debatable as it is a fact. By this we can understand that model can differentiate between opinions and facts.

then the evidence retrieval modules then scrapes the web several times to get relevant data for debatable claims so that we will have both positive and counter statements for that particular statement. By this system becomes context aware of domain

later the argument generation module gives different perspectives both supporting and opposing . The generated arguments are different and aligned with evidence

overall system successfully all NLP components in a single pipeline to make ground the debatable statements along with simplifying text and giving info of popular terms

However , we faced some challenges like the quality of argumwents and evidence depends on pretrained models and external sources so there can be irrelevant data.

finally , the results validate the effectiveness of the proposed system and makes it a powerful tool

Input Paragraph

Artificial intelligence will replace most human jobs within the next 20 years.

NASA launched the James Webb Space Telescope in 2021 to study distant galaxies and exoplanets using infrared technology. According to UN reports, global renewable energy investment reached \$500 billion in 2023.

Analyze Text

Extracted Claims

[1] Artificial intelligence will replace most human jobs within the next 20 years

[2] NASA launched the James Webb Space Telescope in 2021 to study distant galaxies and exoplanets

[3] Global renewable energy investment reached \$500 billion in 2023

Simplified Claims

[1]
Original: Artificial intelligence will replace most human jobs within the next 20 years
Simplified: Artificial intelligence will replace most human jobs within the next 20 years

[2]
Original: NASA launched the James Webb Space Telescope in 2021 to study distant galaxies and exoplanets
Simplified: NASA (The National Aeronautics and Space Administration (NASA) is an independent agency of the U.S. federal government responsible for the United States' civil space program and for research in aeronautics and space exploration.) launched the James Webb Space Telescope (The James Webb Space Telescope (JWST) is a space telescope designed to conduct infrared astronomy. It is the largest telescope in space, and is equipped with high-resolution and high-sensitivity instruments, allowing it to view objects too old, distant, or faint for the Hubble Space Telescope.) in 2021 to study distant galaxies and exoplanets

[3]
Original: Global renewable energy investment reached \$500 billion in 2023
Simplified: Global renewable energy investment reached \$500 billion in 2023

Figure 5: Input with claim extraction and claim simplification with wikipedia definition

Debatability [1] Claim: Artificial intelligence will replace most human jobs within the next 20 years Label: debatable [2] Claim: NASA launched the James Webb Space Telescope in 2021 to study distant galaxies and exoplanets Label: non-debatable [3] Claim: Global renewable energy investment reached \$500 billion in 2023 Label: non-debatable
Retrieved Evidence ===== Claim 1 ===== Claim: Artificial intelligence will replace most human jobs within the next 20 years Source: Will AI improve or eliminate jobs? It depends on who you ask. Content: BOSTON — Experts on artificial intelligence and the economy have a wide range of predictions about the impact of AI systems on the labor market. On the pessimistic side, economist Anton Korinek predicted during a recent interview with Harvard Business School's Institute for the Study of Business in ... Source: Will AI improve or eliminate jobs? It depends on who you ask. Content: Korinek predicted that so-called artificial general intelligence (AGI), AI systems with thinking capabilities similar to or greater than humans, could be achieved within as few as five years. That, he warned, could throw the economic and social order into a tailspin, driving down human wage levels a... Source: Will AI improve or eliminate jobs? It depends on who you ask. Content: "As with any technological shift, we expect human workers to transition toward more complex tasks that command higher wages, while simpler, more repetitive work will face higher risk of disruption from AI," she wrote. What remains to be seen is exactly how severe that disruption may be, and how read...

Figure 6: Debatability classification with distillbert and zeroshot

Retrieved Evidence Source: Will AI really decimate human jobs? Tech industry insiders are split ... Content: Still, Jernite said he worries leaders in Corporate America might feel pressured to start making changes simply because they've heard that AI could replace jobs. "And people follow that direction, they fire people, and it doesn't have anything to do with what the technology can or cannot do, but how..." Source: Will AI really decimate human jobs? Tech industry insiders are split ... Content: Small circles in Silicon Valley have for years been discussing how AI will upend the labor market — along with proposals for mitigating that impact, such as universal basic income, an approach that OpenAI CEO Sam Altman has backed. But the conversation has burst onto the public scene in recent month... Source: What Jobs Will AI Replace? Built In Content: It can change the nature of certain jobs by automating repetitive and mundane tasks, freeing up human workers to focus on more challenging problems. However, AI will first replace a large number of routine and entry-level jobs by 2030 and more advanced roles in the decades after. An earlier version ... Source: What Jobs Will AI Replace? Built In
Filtered Evidence ===== Claim 1 ===== Claim: Artificial intelligence will replace most human jobs within the next 20 years - With the rise of artificial intelligence (AI) and automation, there are both advantages and disadvantages to consider. While AI has the potential to revolutionize industries and increase efficiency, i... Source: Will Artificial Intelligence Replace Jobs? Pros and Cons - People have worried that computers will take their jobs for basically as long as they've been around, but those fears have felt more realistic than ever over the past year as artificial intelligence h... Source: Will AI really decimate human jobs? Tech industry insiders are split ... - As artificial intelligence (AI) continues to advance, there is an ongoing debate about whether it will replace human jobs or lead to a new era of collaboration between humans and computers. This colla... Source: Will Artificial Intelligence Replace Jobs? Pros and Cons - BOSTON — Experts on artificial intelligence and the economy have a wide range of predictions about the impact of AI systems on the labor market. On the pessimistic side, economist Anton Korinek predic... Source: Will AI improve or eliminate jobs? It depends on who you ask. - It can change the nature of certain jobs by automating repetitive and mundane tasks, freeing up human workers to focus on more challenging problems. However, AI will first replace a large number of ro... Source: What Jobs Will AI Replace? Built In - It's that they're rising much more slowly than some of the more breathless media coverage might have you believe — which is great news for most of those who think AI-powered technology will soon steal... Source: What Jobs Will AI Replace? Built In

Figure 7: Web Scaping using beautiful soap and filtering the results

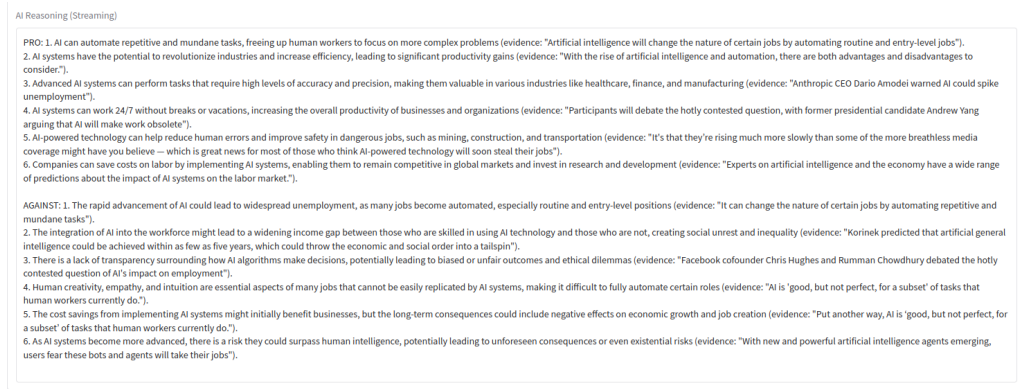


Figure 8: Final Result given by the Mistral model

6 Conclusion

The proposed debate Text simplification chatbot gives a efficient answer for the issue created by typical biased content that is there in internet. models like FlanT5 and mistral understand and give equalized content. Users can get clean information as we combined claim extraction , simplification and explained important names

This chatbot gives supporting and counter arguments along with simplification . by this users can get the whole point about the topic they asked. we get these 2 views by web scraping internet sources and these are given as context to an llm so it will not shows any partiality. By this we will receive correct output points. we then remove unnecessary content that we may get while scraping so that llm doesn't get polluted by this content. finally, by this system we can increase our understanding , have critical thinking and judge proposed statements

References

- [1] C. Raffel *et al.*, "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer," *JMLR*, 2020.
- [2] W. Yin, J. Hay, and D. Roth, "Benchmarking Zero-shot Text Classification: Datasets, Evaluation and Entailment Approach," 2019.
- [3] A. Williams, N. Nangia, and S. Bowman, "A Broad-Coverage Challenge Corpus for Sentence Understanding through Inference," 2018.
- [4] J. Lawrence and C. Reed, "Argument Mining: A Survey," 2019.
- [5] V. Sanh *et al.*, "DistilBERT: a distilled version of BERT," 2020.
- [6] P. Lewis *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," 2020.
- [7] V. Karpukhin *et al.*, "Dense Passage Retrieval for Open-Domain Question Answering," 2020.
- [8] A. Q. Jiang *et al.*, "Mistral 7B," 2023.